RAMDEOBABA UNIVERSITY, NAGPUR

Formerly Shri Ramdeobaba College of Engineering & Management (RCOEM) Est. 1984

Electronics and Communication Engineering

Cloud Provider Comparison

Course	Semester	Total Marks	Group
Agentic AI	Sem IV · B.E. ECE	50 Marks	Team of 2

Guide: Lokesh Dange

Group Members:

- Tanushree Vaishnav (08)
- Arya Nandankar (113)

1. Problem Statement

Organisations evaluating cloud infrastructure face a fragmented and overwhelming landscape. AWS, Microsoft Azure, and Google Cloud Platform each offer hundreds of services with complex pricing models, regional availability differences, and rapidly evolving feature sets. A startup looking for cost-effective compute for a machine learning workload needs entirely different guidance than an enterprise seeking a globally distributed database with strict compliance requirements.

Manual comparison is tedious and error-prone: pricing calculators differ across providers, documentation is scattered across dozens of pages, and community feedback on real-world performance is buried in forums and blog posts. Decision-makers often rely on outdated reports or vendor-biased whitepapers — leading to suboptimal choices that cost thousands of dollars monthly

2. The Solution

Build an AI agent system that takes basic inputs about a workload — compute type, storage needs, budget range, compliance requirements, and geographic preferences — and automatically researches, compares, and recommends the best cloud provider and service configuration. The agent system will:

- Research AWS offerings via Tavily web search — gathering pricing, features, limitations, and community feedback for relevant services
- Research Azure and GCP offerings independently — ensuring unbiased parallel investigation
- Query an internal cloud services knowledge base of vetted pricing benchmarks and feature matrices
- Synthesise findings into a structured comparison covering cost, performance, scalability, compliance, and ecosystem fit
- Generate a recommendation report with a clear verdict, trade-off analysis, and implementation roadmap
- Evaluate output quality via an LLM-as-Judge against a cloud-decision rubric

3. Why Agentic Architecture?

A single prompt cannot simultaneously research live pricing across three providers, query a vetted knowledge base, reason about workload-specific trade-offs, and write a comprehensive comparison report. An agentic pipeline assigns each responsibility to a specialist agent — enabling autonomous, multi-step reasoning that adapts to the user's unique workload and budget constraints.

User	Need	Why Agentic?
CTO / DevOps Lead / Startup Founder	Unbiased, data-driven cloud provider recommendation — fast and well-researched	Multi-step: parallel research + knowledge base + trade-off reasoning + structured writing

4. Task Decomposition & Agent Specs

The pipeline is broken into five sequential specialist agents plus one evaluator agent. Each agent has a clearly defined responsibility, discrete inputs and outputs, and specific tools or models it uses.

Agent	Responsibility	Input	Output	Tools / Model
Agent 1: AWS Researcher	Searches the web for AWS services, pricing, features, limitations, and	Workload type, budget, compute/storage needs, region	AWS resource findings (structured text)	Tavily Search, Gemini Text

Agent	Responsibility	Input	Output	Tools / Model
	community feedback.			
Agent 2: Azure/GCP Researcher	Independently searches for Azure and GCP offerings — pricing, features, and community sentiment.	Workload type, budget, compute/storage needs, region	Azure + GCP findings (structured text)	Tavily Search, Gemini Text
Agent 3: Cloud KB Fetcher	Queries internal vetted knowledge base using function calling.	Workload type + provider names from Agents 1-2	Vetted cloud data snippets (JSON)	Cloud KB Tool, Gemini Function Calling
Agent 4: Recommendation Writer	Synthesises web research + KB data into a structured comparison report.	AWS findings + Azure/GCP findings + KB data	Full Markdown comparison report	Gemini Text (multi-shot)
Agent 5: LLM-as-Judge	Evaluates the report against a 5-criterion cloud-decision rubric.	Full report + original inputs	JSON scores + reasoning + overall score	Gemini (separate call)

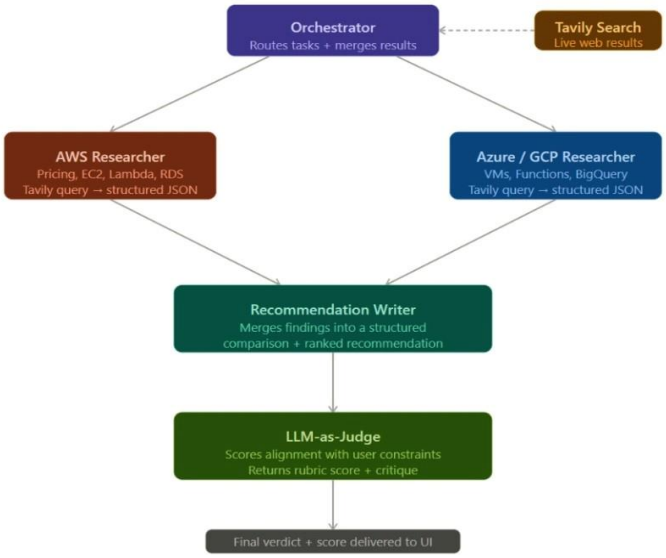
5. Decision Points & Failure Handling

Agent	Decision Points	Failure Handling
AWS Researcher	Decides how many searches to run based on workload complexity; stops when sufficient data found (up to 10 turns)	Returns partial research if max turns reached; flags incomplete data
Azure/GCP Researcher	Model decides which service categories to research based on workload type	Returns partial findings with warning if search yields insufficient results
Cloud KB Fetcher	Reasons about which knowledge base categories to query	Returns empty dataset with warning if KB query fails; continues with web-only data
Recommendation Writer	Adapts recommendation depth based on budget sensitivity and workload criticality	Generates partial report if input data is incomplete; clearly marks missing sections
LLM-as-Judge	Evaluates independently; scores strictly on output quality and decision soundness	Falls back to partial scoring if JSON parse fails; logs error

6. System Architecture

The system runs as a Streamlit web application. All LLM calls are routed through a single genai.Client connected to the Gemini API. Each agent is a Python function that accepts structured inputs, calls the client with a purpose-built prompt, processes the response, and passes structured data to the next agent. The image represents an Agentic AI system architecture designed for cloud provider comparison. At the top, the Orchestrator acts as the central controller that manages the entire workflow by routing tasks to different agents and combining their results. It also interacts with Tavily Search, which provides real-time web data such as pricing, features, and updates. Below this, two research agents work in parallel: the AWS Researcher, which collects detailed information about AWS services like EC2, Lambda, and RDS, and the Azure/GCP Researcher, which gathers similar data for Microsoft Azure and Google Cloud Platform. Both agents independently fetch and structure their findings to ensure unbiased and comprehensive research.

The outputs from these research agents are then passed to the Recommendation Writer, which synthesizes all the collected data into a structured comparison report, including cost analysis, feature comparison, and a final recommendation. After this, the LLM-as-Judge evaluates the generated report based on criteria such as accuracy, completeness, relevance, and reasoning quality, and provides a score along with feedback. Finally, the system delivers the result to the user interface, presenting a clear comparison, recommended cloud provider, and evaluation score. Overall, this architecture demonstrates how multiple specialized AI agents collaborate step-by-step to produce a more accurate and reliable decision compared to a single AI model.



Tech Stack

Layer	Technology	Purpose
Language	Python 3.11+	Agent logic, orchestration, tool execution
UI / Frontend	Streamlit	Input form, pipeline progress, report display
LLM Provider	Gemini API (google-genai SDK)	Text generation, reasoning, evaluation
Search Tool	Tavily Search API	Live web research on cloud provider offerings
Cloud Knowledge Base	JSON file (cloudserviceskb.json)	Vetted pricing benchmarks, feature matrices, compliance data
Deployment	Vercel / Railway	Live public URL for the application
Environment	python-dotenv (.env file)	Secure API key management
Video	Loom	3-minute project walkthrough recording

7. Data Flow — End to End

USER INPUT: Workload Type · Budget Range · Compute Needs · Storage Needs · Region

- AGENT 1 — AWS Researcher: Tavily Search → AWS pricing, features, limitations, community feedback
- AGENT 2 — Azure/GCP Researcher: Tavily Search → Azure + GCP pricing, features, community sentiment
- AGENT 3 — Cloud KB Fetcher: Function Calling → Pricing benchmarks, feature matrices, compliance data
- AGENT 4 — Recommendation Writer: Synthesise all data → Structured comparison report with final recommendation
- AGENT 5 — LLM-as-Judge: Evaluate report against cloud-decision rubric → JSON scores + reasoning

STREAMLIT OUTPUT: Full comparison report · Judge scores · Download button · Provider verdict

8. Project File Structure

```
project/
├── app.py                          # Streamlit UI — entry point, form, orchestration
├── agents/
│   ├── awsresearcher.py            # Agent 1: Tavily search loop for AWS
│   ├── azuregcpresearcher.py       # Agent 2: Tavily search loop for Azure + GCP
│   ├── cloudkbfetcher.py           # Agent 3: Function calling against cloud KB
│   ├── recommendationwriter.py     # Agent 4: Synthesise into comparison report
│   └── judgeagent.py               # Agent 5: LLM-as-Judge evaluation
├── data/
│   └── cloudserviceskb.json        # Vetted cloud pricing benchmarks and features
├── requirements.txt
├── .env                            # API keys (NOT committed to git)
└── README.md                       # Project overview + deployed URL + Loom link
```

9. API Connection — Gemini

The entire pipeline uses a single genai.Client from the google-genai SDK, initialised once in app.py and passed to every agent function.

Client Initialisation (app.py)

```
import os
from google import genai
from dotenv import load_dotenv

load_dotenv()
GEMINI_KEY = os.getenv('GEMINI_API_KEY')
client = genai.Client(api_key=GEMINI_KEY, vertexai=False)
```

Agent Model Usage

Agent	Model Used	Response Field
Agent 1 — AWS Researcher	gemini-2.0-flash + Tavily tool	response.text (research summary)
Agent 2 — Azure/GCP Researcher	gemini-2.0-flash + Tavily tool	response.text (research summary)
Agent 3 — Cloud KB Fetcher	gemini-2.0-flash + function calling	function_call.args (cloud data JSON)

Agent	Model Used	Response Field
Agent 4 — Recommendation Writer	gemini-2.0-flash	response.text (Markdown report)
Agent 5 — LLM-as-Judge	gemini-2.0-flash (separate call)	response.text parsed as JSON scores

10. LLM-as-Judge — Quality Evaluation

After the four pipeline agents produce the comparison report, a fifth separate agent evaluates the output against a structured cloud-decision rubric. The judge has no knowledge of how the report was created, preventing self-bias. It scores the report purely on output quality and decision soundness.

Evaluation Rubric — 5 Criteria

Criterion	What to Check	1 — Poor	3 — Good	5 — Excellent
Pricing Accuracy	Are cost estimates based on current pricing data?	Outdated or fabricated pricing	Roughly accurate but missing key cost factors	Current pricing with all relevant factors; clear monthly estimates
Feature Relevance	Are compared features relevant to the stated workload?	Generic feature list not matched to workload	Some relevant features highlighted	Every feature comparison directly addresses workload requirements
Constraint Alignment	Does the recommendation fit the budget and compliance requirements?	Exceeds budget or ignores compliance	Mostly aligned with minor gaps	Perfectly aligned with budget; all compliance requirements addressed
Completeness	Does the report cover all three providers, cost, features, trade-offs, and recommendation?	Multiple key sections missing	All sections present but some poorly developed	All sections present, well-developed, logically ordered
Reasoning Quality	Is the final recommendation well-justified with trade-off analysis?	No reasoning provided	Some justification but missing trade-offs	Clear reasoning; trade-offs explicitly weighed; decision defensible

11. Sample Pipeline Output

Input: Workload Type: Machine Learning Training | Budget: $500-1000/month | Compute: GPU instances (4x V100 equivalent) | Storage: 500GB high-performance | Region: US East | Compliance: SOC 2

Cost Comparison

Provider	GPU Instance (Monthly)	Storage (500GB)	Data Transfer	Total Estimate
AWS	$720 (p3.2xlarge)	$57.50 (gp3)	$45	$822.50
Azure	$684 (NC6s v3)	$76.80 (Premium SSD)	$43	$803.80
GCP	$548 (n1-standard-8 + T4)	$85 (SSD PD)	$40	$673.00

Feature Comparison

Feature	AWS	Azure	GCP
GPU Options	V100, A100, T4	V100, A100, T4	V100, A100, T4, TPU
ML Platform Integration	SageMaker	Azure ML	Vertex AI
Preemptible/Spot Pricing	Yes (up to 90% off)	Yes (up to 80% off)	Yes (up to 91% off)
SOC 2 Compliance	Yes	Yes	Yes
US East Availability	Yes	Yes	Yes

Recommendation

Primary Recommendation: Google Cloud Platform

For this ML training workload with budget constraints, GCP offers:

- 25% lower cost than AWS for comparable GPU compute
- Native Vertex AI integration for end-to-end ML workflows
- Aggressive preemptible pricing ideal for training jobs that can tolerate interruption
- SOC 2 compliance for security requirements

12. Sample Judge Evaluation

Criterion	Score	Reasoning
Pricing Accuracy	4/5	Current pricing cited with specific instance types. Minor gap: egress costs estimated but not broken down by expected data volume.
Criterion	**Score**	**Reasoning**
Feature Relevance	5/5	Every feature compared directly addresses ML training needs — GPU options, ML platforms, preemptible pricing, compliance.
Constraint Alignment	5/5	Recommendation ($673/mo) fits within $500-1000 budget. SOC 2 compliance explicitly confirmed.
Completeness	5/5	All sections present: executive summary, cost comparison, feature comparison, trade-offs, recommendation, and implementation roadmap.
Reasoning Quality	4/5	Clear justification for GCP recommendation. Could improve by quantifying performance benchmarks for ML training times.
Overall Score	4.6/5.0	Strong, well-reasoned report. Recommendation clearly fits constraints with good trade-off analysis.